

梯度下降算法

[梯度下降算法.pdf](#)

本章的大概路线：

- 推荐系统与矩阵补全
 - 协同过滤与矩阵分解
 - 无约束优化的**迭代方法**
 - **下降方向与步长**
 - **收敛速度与梯度下降**
 - 二次函数**特例**
 - 光滑强凸情形
-

协同过滤与矩阵分解：

路线一：邻域模型

如果两部电影很相似，那么用户对其中一部的评分，可以帮助预测他对另一部的评分。

比如：

- 你喜欢《复仇者联盟》；
- 《钢铁侠》和它很相似；
- 那系统可能预测你也喜欢《钢铁侠》。

这就是基于物品相似性的协同过滤。

路线一：邻域模型（加权协同过滤）

$$\hat{r}_{xi} = b_{xi} + \sum_{j \in N(i;x)} w_{ij}(r_{xj} - b_{xj})$$

► 基线估计：

$$b_{xi} = \mu + b_x + b_i$$

- μ : 全局平均评分
- b_x : 用户 x 的评分偏差 = (用户 x 的平均评分) - μ
- b_i : 电影 i 的评分偏差 = (电影 i 的平均评分) - μ

► 公式的结构：

- 先用 b_{xi} 给出“不看邻居电影时”的基础预测；
- 再用用户 x 对相似电影 j 的评分残差来修正目标电影 i 的预测。

16 / 64

在邻域模型中：

- x : 当前用户；
- i : 目标电影，即想预测评分的电影；
- j : 邻居电影，即用户 x 已经评分过且与目标电影 i 相似的电影；
- $N(i;x)$: 用户 x 已评分电影中，与目标电影 i 相似的一小部分电影。

定义：

$$N(i;x) = \{j : (x,j) \in \Omega_{\text{train}}, j \text{ 与 } i \text{ 相似}\}$$

实际计算时通常只取相似度最高的前 k 个邻居，避免邻域过大。

路线二：矩阵分解

用户和电影背后都有一些隐藏特征。用户喜欢什么类型，电影属于什么类型，两者匹配程度决定评分。

比如电影可以有一些隐含维度：

- 科幻程度、爱情程度、动作程度、搞笑程度、文艺程度。

用户也有类似偏好向量。如果用户偏好向量和电影属性向量很匹配，预测评分就高。核心公式是：

$$\hat{r}_{xi} = q_i^T p_x = \sum_{f=1}^r q_{if} p_{xf}$$

► 直观理解： r 越大表达能力越强，但计算成本和过拟合风险也更高。

	1	-4	2
items	-5	6	5
	-2	3	5
	1.1	2.1	3
	-7	2.1	-2
	-1	7	3
f factors	Q		

	1.1	-2	3	5	-2	-5	8	-4	3	1.4	2.4	-9
f factors	-8	7	5	1.4	3	-1	1.4	2.9	-7	1.2	-1	1.3
	2.1	-4	6	1.7	2.4	9	-3	4	8	7	-6	1
	PT											
	users											

为了防止过拟合并限制参数规模，通常在目标函数中加入正则项(也可以理解为是惩罚项，让参数不要飘)：

$$\min_{P, Q} \sum_{(x, i) \in \Omega_{\text{train}}} (r_{xi} - q_i^T p_x)^2 + \lambda \left[\sum_x \|p_x\|_2^2 + \sum_i \|q_i\|_2^2 \right]$$

其中：

$$\sum_x \|p_x\|_2^2 + \sum_i \|q_i\|_2^2 = \|P\|_F^2 + \|Q\|_F^2$$

正则项惩罚过大的用户向量和电影向量，降低模型为了拟合少量观测评分而过拟合的风险。同时，由于矩阵分解存在尺度不唯一性：

$$q_i^T p_x = (cq_i)^T (p_x/c)$$

正则化也可以缓解参数大小失控的问题，使模型更加稳定。

矩阵分解的目标函数为：

$$F(P, Q) = \sum_{(x, i) \in \Omega_{\text{train}}} (r_{xi} - q_i^T p_x)^2 + \lambda \left[\sum_x \|p_x\|_2^2 + \sum_i \|q_i\|_2^2 \right]$$

其中 p_x 是用户 x 的隐向量， q_i 是电影 i 的隐向量。梯度下降通过如下方式更新参数：

$$P^{k+1} \leftarrow P^k - \tau \nabla_P F(P^k, Q^k)$$

$$Q^{k+1} \leftarrow Q^k - \tau \nabla_Q F(P^k, Q^k)$$

对电影向量 q_i ，其第 f 个分量的梯度为：

$$(\nabla_{q_i} F)_f = -2 \sum_{x \in \Omega_i} (r_{xi} - q_i^T p_x) p_{xf} + 2\lambda q_{if}$$

对用户向量 p_x ，其第 f 个分量的梯度为：

$$(\nabla_{p_x} F)_f = -2 \sum_{i: (x,i) \in \Omega_{\text{train}}} (r_{xi} - q_i^T p_x) q_{if} + 2\lambda p_{xf}$$

梯度由两部分构成：第一部分来自预测误差，第二部分来自正则化项。预测误差推动模型拟合训练数据，正则化项限制参数过大，降低过拟合风险。

矩阵分解的损失函数可以按已知评分样本拆开：

$$F(P, Q) = \sum_{(x,i) \in \Omega_{\text{train}}} F_{xi}(P, Q)$$

其中：

$$F_{xi} = \frac{1}{2} (r_{xi} - q_i^T p_x)^2 + \frac{\lambda}{2} (\|p_x\|_2^2 + \|q_i\|_2^2)$$

这说明总损失是样本级可加的。 SGD 的思想是：每次随机抽取一个评分样本 (x, i) ，用该样本的梯度近似总体梯度，并只更新对应的用户向量 p_x 和电影向量 q_i 。

虽然同一个 p_x 会出现在用户 x 的所有评分中，同一个 q_i 会出现在电影 i 的所有评分中，因此变量不完全可分；但每个样本只涉及局部参数 p_x, q_i ，所以 SGD 每步计算成本很低，适合大规模稀疏评分矩阵。

无约束优化的梯度下降算法：

下降方向与步长：

首先我们来看迭代方法的核心公式：

$$x^{k+1} = x^k + \alpha_k d^k$$

这个公式就是所有迭代优化算法的**共同骨架**。

我们现在站在 x^k 这个位置，接下来决定往哪个**方向**走 d^k ，再决定**走多远** α_k ，于是到达新位置 x^{k+1} 。

梯度下降方向：

$$d^k = -\nabla f(x^k)$$

即沿负梯度方向下降。这是**函数值下降最快**的方向，理解为我们选了一个最陡的坡。

牛顿方向：

$$d^k = -(\nabla^2 f(x^k))^{-1} \nabla f(x^k)$$

其中 $\nabla^2 f(x^k)$ 是 Hessian 矩阵。若 Hessian 正定，则牛顿方向是下降方向。海森矩阵能够描述函数在当前点附近的“弯曲程度”“曲率信息”。

拟牛顿方向：

$$d^k = -H_k \nabla f(x^k), \quad H_k \succ 0$$

其中 H_k 用来近似 Hessian 逆矩阵。（因为算海森矩阵的工作量实在是太大了），如果说牛顿法是真实精确的曲率，那么拟牛顿方向就是近似曲率。

迭代优化算法每一步都可以看成：从当前点 x^k 出发，**选择一个下降方向 d^k** ，再**配合步长 α_k** ，得到下一个点 x^{k+1} 。梯度下降、牛顿法和拟牛顿法的区别主要在于**搜索方向**的选择不同。

如何确定步长呢？

精确线搜索是在当前方向 d^k 上寻找**使函数值最小**的步长：

$$f(x^k + \alpha_k d^k) = \min_{\alpha \geq 0} f(x^k + \alpha d^k)$$

即：

$$\alpha_k = \arg \min_{\alpha \geq 0} f(x^k + \alpha d^k)$$

它的思想是：**方向已定，只在这条直线上寻找最优步长**。二分法、黄金分割法等可以用于求解这个**一维搜索问题**。精确线搜索每一步效果较好，但计算成本较高。

Armijo 回溯线搜索

Armijo 规则不要求找到最优步长，只要求**函数值下降得足够多**。

$$s > 0, \quad \beta \in (0, 1), \quad \sigma \in (0, 1)$$

从初始步长 s 开始试探。如果不满足下降条件，则依次缩小为：

$$s\beta, s\beta^2, s\beta^3, \dots$$

$$\text{令 } \alpha = s\beta^m$$

Armijo 条件为：

$$f(x^k + \alpha d^k) \leq f(x^k) + \sigma \alpha \nabla f(x^k)^T d^k$$

$$f(x^k) - f(x^k + \alpha d^k) \geq -\sigma \alpha \nabla f(x^k)^T d^k$$

由于 d^k 是下降方向，有：

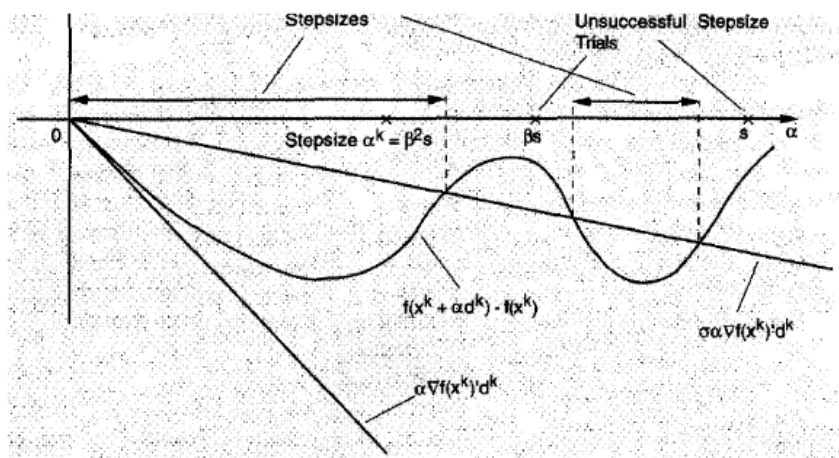
$$\nabla f(x^k)^T d^k < 0$$

所以右侧是正数，表示算法要求实际**下降量达到一定程度**。Armijo 规则的核心思想是：先尝试较大步长，**如果下降不够，就不断缩小步长，直到满足“充分下降”条件**。

参数	含义	常见理解
(s)	初始候选步长	先大胆试多远

参数	含义	常见理解
(β)	缩小比例	不合格时缩短多少
(σ)	充分下降要求	下降多少才算够

► 它不追求一维方向上的最优步长，只要求“下降足够多”，因此比精确线搜索便宜。



Armijo从本质上来看，只是一种**确定步长的方法or原则**，选方向什么的~~不归Armijo管~~，这个用我们说的三种方法（梯度下降、拟牛顿、牛顿）有关

Armijo 是从一个候选**大的步长** s 开始试的，所以如果**下降得不够多**，那应该是走得太远了，已经离开了当前梯度能可靠描述的局部范围，所以我们要**减小步长**。

方法	核心要求	是否要求最优步长	计算代价
精确线搜索	在当前方向上找到使函数值最小的步长	要求最优	通常较高
Armijo 规则	从候选步长开始试，只要函数值下降“足够多”就接受	不要求最优	通常较低

收敛速度与梯度下降：

$$\frac{e_{k+1}}{e_k} \rightarrow \mu, \quad 0 < \mu < 1$$

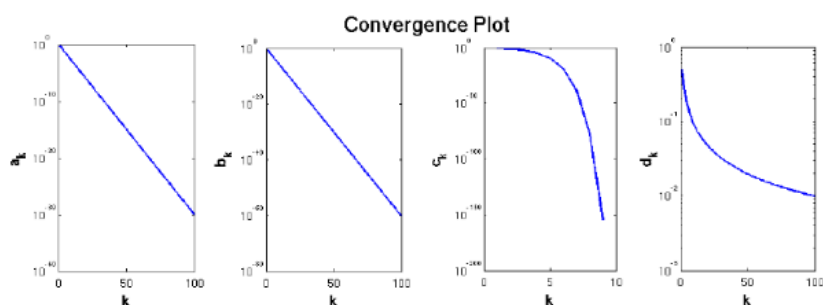
这里 e_k 表示第 k 步的**误差**。它可以是下面这些形式：

- $e_k = \|x^k - x^*\|$
- $e_k = f(x^k) - f(x^*)$
- $e_k = \|\nabla f(x^k)\|$

线性收敛里的“线性”，不是说误差像直线一样下降。真正意思是**每一步误差大约缩小为原来的一个固定比例**。由于他像等比数列一样下降，所以也可以叫做是**几何收敛**。

- ▶ 如果 $\mu \in (0, 1)$ ，则 $\{x^k\}$ 线性收敛。
- ▶ 如果 $\mu = 0$ ，则 $\{x^k\}$ 超线性收敛。
- ▶ 如果 $\mu = 1$ ，通常不能判定为线性收敛；常见情形是次线性收敛，需要进一步比较 $e(x^k)$ 与 $1/k^p$ 等序列。

- ▶ $a_k = 1/2^k$: $\frac{a_{k+1}}{a_k} \rightarrow \frac{1}{2}$ ，线性收敛， $\mu = \frac{1}{2}$ 。
- ▶ $b_k = 1/4^{\lfloor k/2 \rfloor}$: 比值在 1 和 $\frac{1}{4}$ 之间振荡，不能用简单极限 μ 描述，但总体仍呈几何下降。
- ▶ $c_k = 1/2^{2^k}$: $\frac{c_{k+1}}{c_k} \rightarrow 0$ ，超线性收敛，下降非常快。
- ▶ $d_k = 1/(k+1)$: $\frac{d_{k+1}}{d_k} \rightarrow 1$ ，典型次线性收敛，需要看 $O(1/k)$ 这类复杂度。



从迭代次数 k 到误差的换算：

- ▶ 如果误差满足 $e_k \leq C\mu^k$ ，其中 $0 < \mu < 1$ ，要达到 $e_k \leq \epsilon$ ，只需

$$k \geq \frac{\log(C/\epsilon)}{\log(1/\mu)}.$$

因此线性收敛对应 $O(\log(1/\epsilon))$ 次迭代。

- ▶ 如果目标函数误差满足 $f(x^k) - f(x^*) \leq C/k$ ，要达到 ϵ 精度，需要

$$k \geq C/\epsilon,$$

即 $O(1/\epsilon)$ 。

- ▶ 如果 $f(x^k) - f(x^*) \leq C/k^2$ ，则需要 $k \geq \sqrt{C/\epsilon}$ ，即 $O(1/\sqrt{\epsilon})$ 。
- ▶ 所以“关于 k 的收敛率”和“达到 ϵ 精度的迭代复杂度”可以互相换算。

梯度下降与二次函数特例

梯度下降算法

前面我们一直在科普梯度下降、其他选方向方法，确定步长方法，搭建了一个**通用的框架**，现在我们正式进入梯度下降算法：

方向具体为负梯度方向：

$$d^k = -\nabla f(x^k)$$

于是得到梯度下降：

$$x^{k+1} = x^k - \alpha_k \nabla f(x^k)$$

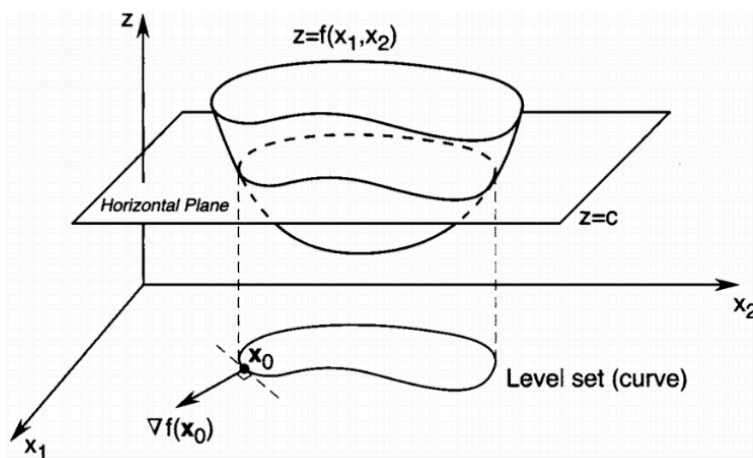
从几何意义上来看，梯度**垂直于**通过 x_0 的函数的水平及曲线的**切线**，指向函数值增长最快的方向。

假设我们沿着这条水平集曲线走。因为你一直在**同一条水平线上**，所以**函数值不变**。也就是说，如果 t 是这条曲线在 x_0 处的切线方向，那么**沿着 t 方向走**，函数值**变化率应该是 0**。而沿方向 t 的**变化率是方向导数**：

$$\nabla f(x_0)^T t$$

因为沿水平集走函数值不变，所以： $\nabla f(x_0)^T t = 0$ 。两个向量内积为 0，说明它们垂直。所以：

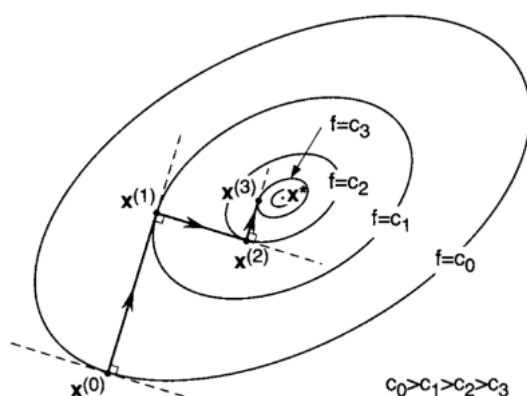
$$\nabla f(x_0) \perp t$$



最速下降法：(=负梯度方向 + 精确线搜索)

- 方向仍然是： $-\nabla f(x^k)$
- 步长不是随便取，而是在这个方向上**找最优的**： α_k
- 具体地：

$$\alpha_k = \arg \min_{\alpha \geq 0} f(x^k - \alpha \nabla f(x^k))$$



二次规划的最速下降：

- 最速下降迭代：从任意 $x^{(0)}$ 开始，设置

$$x^{(k+1)} = x^{(k)} - \alpha_k g^{(k)}, \quad k = 0, 1, 2, \dots$$

其中 $g^{(k)} := \nabla f(x^{(k)})$ 且

$$\alpha_k = \arg \min_{\alpha \geq 0} f(x^{(k)} - \alpha g^{(k)}) = \frac{g^{(k)T} g^{(k)}}{g^{(k)T} Q g^{(k)}}$$

- 收敛速度：由初始点和 Q 的谱条件数决定。为了分析它们，我们

- 定义解误差： $e^{(k)} = x^{(k)} - x^*$
- 具有性质： $g^{(k)} = Qx^{(k)} - b = Qe^{(k)}$ 。

所以我们看到梯度其实可以由误差通过矩阵变换得来。具体过程如下：

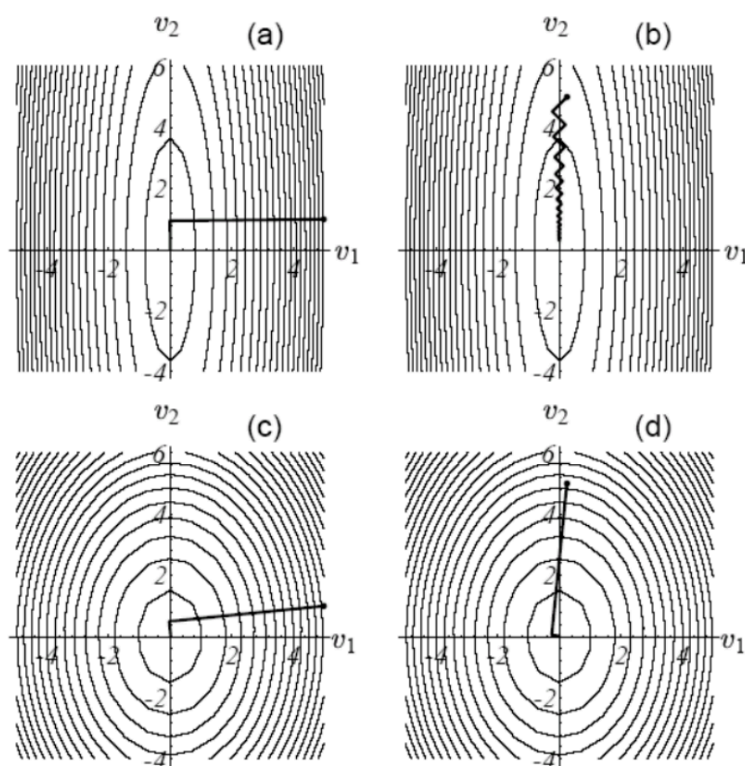
定义当前点和最优点的误差： $e^k = x^k - x^*$ （注意这里的误差是误差向量哦，所以既有大小也有方向）。因为最优点满足： $Qx^* = b$ 。所以： $g^k = Qx^k - b = Qx^k - Qx^* = Q(x^k - x^*)$ 。

因此: $g^k = Qe^k$ 。

如果 Q 把 $e^{(k)}$ 变换之后, 方向没有改变, 只是放大了一个倍数, 那么梯度方向就和误差方向一致。什么时候矩阵 Q 作用在一个向量上, 只改变长度, 不改变方向? 答案就是这个**向量是矩阵的特征向量**时。

当误差向量 $e^{(k)}$ 是 Q 的特征向量时, 负梯度方向正好指向最优点。其实很好理解, 因为误差向量本来就是最优解-当前解算出来的, 方向肯定是**最优解**→**当前解**, 如果我们的梯度和这个方向**平行**, 从这个点出发, 那肯定是**指向最优解**的, 也就是**指回去**。

一句话总结, 如果误差方向是矩阵的特征向量, 那么负梯度方向指向最优解, 配合精确线搜索, 一步登天。



特征向量方向是 Q 不会扭转方向的方向。

如果当前误差刚好在这种方向上, 那么梯度方向和误差方向平行, 负梯度就能直接指向最优点。

如果当前误差不是这种方向, Q 会把误差方向扭一下, 负梯度就不会直接指向最优点, 于是可能绕着走、之字形走。

光滑强凸情形:

这里的“光滑”不是说图像看起来圆滑，而是说：函数不要弯得太突然，梯度变化不要太剧烈。数学上通常写成：

∇f 是 L -Lipschitz

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|$$

直观理解就是我们从 x 走到 y ，梯度可以变，但不能突然暴跳。在二阶可微时，光滑性可以理解成：

$$\nabla^2 f(x) \preceq MI$$

也就是说函数的最大弯曲程度被 M 控制住。

而强凸性质就是函数像一个“有一定弯曲程度的碗”，不会太平。数学上写成：

$$mI \preceq \nabla^2 f(x)$$

意思是：函数在任何方向上的弯曲程度至少有 m 这么多。

所以既光滑又强凸其实就是要求函数**既不能太平，也不能太弯**。其中：

- m ：最小曲率，下界；
- M ：最大曲率，上界。

如果 m 和 M 差不多，说明这个碗比较圆润。如果 M 远大于 m ，说明这个碗很狭长，梯度下降会慢。

所以我们定义**条件数** $\kappa = \frac{M}{m}$ 。前面在二次函数那里也见过类似的东西： $\kappa = \frac{\lambda_{\max}(Q)}{\lambda_{\min}(Q)}$ 。在一般函数里， M 类似最大曲率， m 类似最小曲率。所以：

$$\kappa = \frac{M}{m}$$

表示：不同方向的弯曲程度差异有多大。接近圆形下降快，狭长的话容易走之“字”，就下降慢。条件数越大，梯度下降越慢。

我们还要记住一个结论：**梯度下降在光滑强凸情形下满足线性收敛：**

$$f(x^k) - f(x^*) \leq \left(1 - \frac{m}{M}\right)^k (f(x^0) - f(x^*))$$

对应迭代复杂度为：

$$O(\log(1/\varepsilon))$$